
Manual Técnico — Espaço Terapia OS

Versão do documento: gerado a partir do repositório monorepo `espaco-terapia-os`

Público-alvo: engenheiros de software, DevOps, arquitetos, auditoria técnica

Formato: Markdown modular + PDF (`MANUAL-TECNICO.pdf`)

Como usar este manual

1. Leia a **Parte I (Fundamentos)** para arquitetura, stack, configuração, logs e API.
 2. Consulte **Parte II (Módulos)** alinhada ao menu lateral da aplicação.
 3. Use **Apêndices** para glossário, matriz de integração e rotas auxiliares.
 4. Contratos HTTP detalhados: Swagger UI (`SWAGGER_ENABLED=true`) em `/api/v1/swagger/index.html`.
-

Parte I — Fundamentos

#	Capítulo	Arquivo
00	Introdução e visão do sistema	manual-tecnico/00-introducao.md
01	Stack tecnológico	manual-tecnico/01-stack.md
02	Arquitetura backend	manual-tecnico/02-arquitetura-backend.md
03	Organização do código	manual-tecnico/03-organizacao-codigo.md
04	Configuração e ambientes	manual-tecnico/04-configuracao.md
05	Logging e observabilidade	manual-tecnico/05-logging.md
06	API REST e Swagger	manual-tecnico/06-api-swagger.md
07	Autenticação e RBAC	manual-tecnico/07-autenticacao-rbac.md
08	Banco de dados (ER)	manual-tecnico/08-banco-dados.md
09	Frontend transversal	manual-tecnico/09-frontend.md
10	DevOps e segurança	manual-tecnico/10-devops-seguranca.md

Parte II — Módulos (menu lateral)

Dashboard

Módulo	Arquivo
Dashboard (/)	modulos/00-dashboard.md

Recepção

Módulo	Rota	Arquivo
Agenda	/agenda	modulos/01-agenda.md
Agendamentos	/consultas	modulos/02-agendamentos.md
Pacientes	/pacientes	modulos/03-pacientes.md
Terapias	/terapias	modulos/04-terapias.md
Salas	/salas	modulos/05-salas.md

Clínico

Módulo	Rota	Arquivo
Prontuários	/prontuarios	modulos/06-prontuarios.md
Anamneses	/anamneses	modulos/07-anamneses.md
Aprovação Atendimentos	/atendimentos/aprovacoes	modulos/08-aprovacao-atendimentos.md
Docs Assinados	/documentos-assinados	modulos/09-docs-assinados.md

Profissionais

Módulo	Rota	Arquivo
Meu Painel	/meu-painel	modulos/10-meu-painel.md
Minha Agenda	/minha-agenda	modulos/11-minha-agenda.md
Profissionais	/profissionais	modulos/12-profissionais.md

Financeiro

Módulo	Rota	Arquivo
Financeiro	/financeiro	modulos/13-financeiro.md
Balancetes	/balancetes	modulos/14-balancetes.md
Relatórios Conciliação	/relatorios-conciliacao	modulos/15-relatorios-conciliacao.md
Auditoria de Notas	/auditoria-notas	modulos/16-auditoria-notas.md

RH & Contratos

Módulo	Rota	Arquivo
Contratos	/contratos	modulos/17-contratos.md
Folha de Pagamento	/folha-pagamento	modulos/18-folha-pagamento.md

Planos & Jurídico

Módulo	Rota	Arquivo
Planos de Saúde	/planos-saude	modulos/19-planos-saude.md
Ações Judiciais	/acoes-judiciais	modulos/20-acoes-judiciais.md

Estoque & Ativos

Módulo	Rota	Arquivo
Estoque	/estoque	modulos/21-estoque.md
Comodato	/comodato	modulos/22-comodato.md

Relatórios

Módulo	Rota	Arquivo
Relatórios	/relatorios	modulos/23-relatorios.md
Relatórios Avançados	/relatorios-avancados	modulos/24-relatorios-avancados.md

Marketing e Documentos

Módulo	Rota	Arquivo
Marketing	/marketing	modulos/25-marketing.md
Documentos	/documentos	modulos/26-documentos.md

Administração

Módulo	Rota	Arquivo
Usuários	/configuracoes/usuarios	modulos/27-usuarios.md
Controles de acesso	/configuracoes/controles-acesso	modulos/28-controles-acesso.md
Chave Digital	/configuracoes/chave-digital	modulos/29-chave-digital.md

Parte III — Apêndices

Apêndice	Arquivo
A — Glossário	apendices/glossario.md
B — Matriz integração API	apendices/matriz-integracao.md
C — Rotas auxiliares	apendices/rotas-auxiliares.md

Documentação operacional relacionada

- [DEV.md](#) — desenvolvimento local e E2E
- [DEPLOY.md](#) — produção
- [SECURITY.md](#) — políticas de segurança
- [backend/README.md](#) — API e seeds

Regenerar o PDF

Pré-requisitos (um dos cenários):

- **Pandoc** + LaTeX (pdf_latex ou xelatex), ou
- **Node.js** + pacote md-to-pdf (instalado automaticamente pelo script)

Comando:

```
npm run docs:manual:pdf
```

Saída: docs/MANUAL-TECNICO.pdf

O script concatena todos os capítulos em ordem, pré-renderiza diagramas Mermaid quando @mermaid-js/mermaid-cli (mmdc) estiver disponível, e gera o PDF.

Atualizar módulos gerados por template:

```
python3 scripts/generate-manual-modules.py
```

Template interno

Novo capítulo de módulo: copiar [manual-tecnico/_TEMPLATE.md](#).

00 — Introdução e visão do sistema

1. Propósito do sistema

Espaço Terapia OS é um sistema de gestão operacional e clínica para **clínica pediátrica multi-filial**. O domínio cobre:

- Cadastro de **unidades** (filiais) e vínculo paciente-unidade
- **Agenda e consultas** (agendamento, confirmação, conclusão, fluxo de atendimento/prontuário)
- **Prontuário eletrônico, anamneses** e aprovação de atendimentos
- **Profissionais** (terapeutas) com documentação obrigatória e pendências
- **Financeiro, contabilidade** (balancete), **RH** (folha CLT/PJ), **contratos** com assinatura/compartilhamento público
- **Planos de saúde, ações judiciais, notas fiscais** e conciliação
- **Estoque, comodato, marketing** (materiais/manuais)
- **Biblioteca de documentos** institucionais e **documentos assinados** (chave digital)

- **RBAC** granular (permissões de menu + escopo de dados por role)

2. Diagrama de contexto (C4 — nível 1)

Diagrama (ver Markdown): [flowchart TB...](#)

3. Diagrama de containers (C4 — nível 2)

Diagrama (ver Markdown): [flowchart LR...](#)

4. Personas e papéis (RBAC)

Role	Descrição típica	Grupos de menu visíveis
admin	Administrador técnico	Todos + Configurações
gestor	Gestão clínica/operacional	Quase todos exceto alguns itens só-admin
funcionario	Recepção / apoio	Recepção, Clínico, Profissionais, Documentos
terapeuta	Profissional de saúde	Meu Pannel, Minha Agenda, módulos clínicos
terceiro	Financeiro/RH externo	Financeiro, RH, Relatórios

Permissões de menu são strings do tipo `menu.{recurso}.view`, avaliadas em `AuthContext.hasPermission()` e em `ProtectedRoute`.

5. Unidade ativa (multi-filial)

O frontend mantém **unidade selecionada** em `UnidadeContext`. Consultas, agenda e listagens filtradas por `unidade_id` (UUID da API). O mapeamento slug local → UUID usa `Unidade.apiId` quando `VITE_API_UNIDADES=true`.

6. Documentação relacionada

Documento	Conteúdo
DEV.md	Setup local, E2E, checklist produção
DEPLOY.md	Deploy Docker, migrations, permissões uploads
SECURITY.md	Políticas de segurança
backend/README.md	API, swagger, seeds

7. Versionamento deste manual

- **Gerado a partir do repositório:** branch de trabalho atual
 - **Regenerar PDF:** `npm run docs:manual:pdf` (ver seção final em [MANUAL-TECNICO.md](#))
 - **API OpenAPI:** `cd backend && make swagger`
-

01 — Stack tecnológico

1. Backend (Go)

Componente	Versão / biblioteca	Função
Linguagem	Go 1.25.0	Runtime (backend/go.mod)
HTTP	gin-gonic/gin v1.10	Router, middleware, binding
ORM	gorm.io/gorm + driver postgres	Persistência
Migrations	golang-migrate (Docker image)	Schema versionado em backend/migrations/
Auth	golang-jwt/jwt v5	Access tokens + revogação (jwt_revocations)
Senha	golang.org/x/crypto (bcrypt)	Hash de credenciais
Docs API	swaggo/swag	OpenAPI 2.0 em backend/docs/
Logs	log/slog (stdlib)	JSON/text estruturado
Erros runtime	getsentry/sentry-go	Panics e tracing opcional
E-mail	Resend API (via env)	Reset de senha, notificações
UUID	google/uuid	Identificadores

Entrypoints:

- backend/cmd/api/main.go — servidor HTTP principal
- backend/cmd/seed-admin/main.go — seed administrador
- backend/cmd/seed-anamneses/main.go — seed anamneses
- backend/cmd/extract-anamneses/main.go — utilitário DOCX

2. Frontend (TypeScript / React)

Componente	Biblioteca	Função
Build	Vite 5.x	Dev server, HMR, proxy /api
UI	React 18	Componentes
Roteamento	react-router-dom v6	SPA routes
Estado servidor	@tanstack/react-query v5	Cache, invalidação, refetch
Formulários	react-hook-form + zod	Validação
UI kit	Radix UI + shadcn/ui	Acessibilidade, primitivos
Estilo	Tailwind CSS	Utility-first
Drag-drop	@dnd-kit	Agenda, kanban onde aplicável
Ícones	lucide-react	Sidebar e ações
Testes unit.	Vitest	src/lib, componentes
E2E	Playwright	e2e/*.spec.ts

3. Infraestrutura

Artefato	Descrição
docker-compose.yml	Serviços api, db, profile migrate
deploy/	Scripts e exemplos produção
Nginx (produção)	Serve SPA estático + reverse proxy API
PostgreSQL 16	Banco relacional único

4. Qualidade e segurança (tooling)

- go test ./..., go vet ./...
- golangci-lint (recomendado em CI)
- eslint no frontend
- backend/scripts/verify-routes.sh — smoke HTTP + contagem Swagger paths

5. Matriz de integração API (frontend)

Cada módulo pode ser desligado via VITE_API_*=false (fallback localStorage legado Lovable):

Flag	Módulo
VITE_API_PACIENTES	Pacientes
VITE_API_PROFISSIONAIS	Profissionais
VITE_API_CONSULTAS	Consultas / Agenda
VITE_API_SALAS	Salas
VITE_API_UNIDADES	Unidades
VITE_API_TERAPIAS	Terapias
VITE_API_ANAMNESES	Anamneses
VITE_API_PRONTUARIO	Prontuário
VITE_API_FINANCEIRO	Financeiro
VITE_API_CONTABILIDADE	Balancetes
VITE_API_RELATORIOS	Relatórios
VITE_API_ESTOQUE	Estoque
VITE_API_COMODATO	Comodato
VITE_API_RH	Folha
VITE_API_PLANOS	Planos / Ações / NF
VITE_API_CONTRATOS	Contratos
VITE_API_MARKETING	Marketing
VITE_API_DOCUMENTOS	Biblioteca documentos
VITE_API_CHAVE_DIGITAL	Chave digital
VITE_API_DOCUMENTOS_ASSINADOS	Docs assinados
VITE_API_AUDIT	Auditoria

Definição: [src/lib/featureFlags.ts](#).

02 — Arquitetura em camadas (backend)

1. Princípio

O backend segue **arquitetura em camadas** com dependência unidirecional:

interfaces → application → domain ← infrastructure

- **domain** não importa Gin nem GORM em interfaces públicas (entities + ports).
- **infrastructure** implementa repositórios e adapters (JWT, storage, postgres).
- **interfaces/http** contém apenas adaptadores HTTP finos.

Comparação com NextBridgeAI: herdado o modelo de camadas e migrations; rejeitado DI monolítico — usa ModuleDeps modular em [routes/deps.go](#).

2. Diagrama de pacotes

Diagrama (ver Markdown): flowchart TB...

3. Bootstrap HTTP

Arquivo central: [backend/internal/interfaces/http/routes/routes.go](#).

Ordem de middleware global:

1. gin.Recovery()
2. middleware.RequestMeta()
3. middleware.AccessLog(cfg, logger) — delega para logger.GinMiddleware
4. middleware.CORS(allowedOrigins)

Grupo /api/v1 público:

- GET/HEAD /health
- POST /auth/login, /auth/forgot-password, /auth/reset-password, /auth/token (bootstrap dev)
- Rotas públicas de contratos (RegisterContratosPublicRoutes)

Grupo **protegido** (JWT obrigatório):

1. RequireConfiguredSecret(JWT_SECRET)
2. RequireAuth(jwtService) — valida Bearer, revogação
3. ActorContext() — user_id, roles no contexto
4. GinUserContext()
5. RequirePasswordChanged() — força troca de senha temporária

Depois: RegisterProtectedRoutes (Wave 1–3).

4. Módulos registrados (Wave 1–3)

Ordem	Register	Domínio
1	register_pacientes	Pacientes
2	register_chave_digital	Chave + docs assinados
3	register_unidades	Unidades (leitura)
4	register_profissionais	Profissionais + documentos
5	register_consultas	Consultas / agenda
6	register_salas	Salas
7	register_notification	Config notificações
8	register_terapias	Terapias
9	register_anamneses	Anamneses + respostas
10	register_prontuario	Prontuário
11	register_financeiro	Financeiro
12	register_relatorios_operacionais	Relatórios
13	register_rh	RH / folha
14	register_estoque	Estoque
15	register_comodatos	Comodato
16	register_planos	Planos, ações, NF
17	register_contratos	Contratos
18	register_marketing	Marketing
19	register_contabilidade	Contabilidade
20	register_audit	Audit log
21	register_users	Usuários
22	register_access_control	RBAC
23	register_documentos	Biblioteca documentos

5. Padrão de resposta HTTP

Sucesso:

```
{
  "data": { },
  "meta": { "page": 1, "per_page": 20, "total": 100 }
}
```

Erro:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Mensagem amigável",
    "details": [{ "field": "email", "message": "inválido" }]
  }
}
```

Implementação: `internal/interfaces/http` (ErrorHandler, helpers de envelope).

6. Guards de permissão por recurso

Cada `register_*.go` define guards com métodos `read`, `write`, `delete` mapeados para permissões RBAC (ex.: `pacientes.read`, `consultas.write`). Admin bypass onde aplicável via `AccessControlService`.

7. Graceful shutdown

[cmd/api/main.go](#): captura SIGINT/SIGTERM, `http.Server.Shutdown` com timeout 15s, flush Sentry.

03 — Organização do código

1. Estrutura do repositório (monorepo)

```
espaco-terapia-os/
├─ backend/           # API Go
│  └─ cmd/api/        # main HTTP
│     └─ internal/
│        └─ application/ # apps (orquestração)
│           └─ config/   # env → Config
│              └─ domain/ # entities, services, repo interfaces
│                 └─ infrastructure/ # postgres, auth, storage, logging
│                    └─ interfaces/http/ # handlers, dto, routes, middleware
```

```

|   |   └─ platform/logger/ # slog, sanitizer, middleware
|   └─ migrations/         # SQL versionado
|   └─ docs/                # Swagger gerado
|   └─ scripts/
└─ src/                     # SPA React
    └─ pages/               # rotas principais
    └─ components/         # UI por domínio
    └─ hooks/              # React Query wrappers
    └─ lib/api/             # client HTTP + módulos
    └─ contexts/           # Auth, Unidade
    └─ lib/mappers/        # DTO ↔ UI
└─ e2e/                    # Playwright
└─ deploy/                 # produção
└─ docs/                   # documentação operacional + manual

```

2. Frontend — convenções

Pasta	Responsabilidade
pages/	Uma página por rota principal; compõe hooks + componentes
hooks/use*.ts	Queries/mutations TanStack Query; chaveia por unidadeId quando necessário
lib/api/*.ts	Funções apiRequest tipadas; types em *.types.ts
components/{dominio}/	Modais, tabelas, formulários específicos
components/layout/	AppLayout, AppSidebar, header
components/common/	FormModal, tabelas genéricas

Rotas: definidas em [src/App.tsx](#). Redirect legado: /tratamentos → /terapias.

Proteção: [ProtectedRoute](#) com requiredPermission opcional.

3. Backend — convenções

Pasta	Responsabilidade
handlers/*_handler.go	Parse HTTP, status codes, chama service
dto/*_request.go	Binding JSON/query; tags validate
domain/service/*_service.go	Regras de negócio, transações lógicas
domain/entity/	Structs de domínio
infrastructure/database/*_model.go	Tags GORM
infrastructure/database/postgres/*_repository.go	SQL/GORM

Nomenclatura HTTP: plural em paths (/pacientes, /consultas).

Soft delete: coluna deleted_at onde migration define; endpoints POST /:id/restore em users, pacientes, profissionais.

4. Migrations

- Arquivos: NNNNNN_descricao.up.sql / .down.sql
- Nunca alterar migration aplicada em produção — criar nova versão
- Trigger padrão: fn_set_updated_at() em tabelas mutáveis

5. Testes — localização

Tipo	Onde
Service unit	backend/internal/domain/service/*_test.go
Handler HTTP	backend/internal/interfaces/http/handlers/*_test.go
Frontend unit	src/**/*_test.ts(x)
E2E	e2e/*.spec.ts

04 — Configuração e ambientes

1. Variáveis backend (config.Config)

Fonte: [backend/internal/config/config.go](#), exemplos em [backend/.env.example](#).

Variável	Obrigatório (prod)	Default dev	Descrição
APP_ENV	sim	development	development production
SERVER_PORT	sim	8080	Porta HTTP
DATABASE_URL	sim	postgres local	DSN PostgreSQL
JWT_SECRET	sim em prod	vazio	HMAC para access token
JWT_ISSUER	não	espaco-terapia-api	Claim iss
JWT_EXPIRATION_MINUTES	não	60	TTL access token
BOOTSTRAP_AUTH_ENABLED	false em prod	true	POST /auth/token dev

Variável	Obrigatório (prod)	Default dev	Descrição
BOOTSTRAP_AUTH_TOKEN	se bootstrap on	—	Header/token bootstrap
CORS_ALLOWED_ORIGINS	sim (lista)	localhost:5173	CSV de origens
SWAGGER_ENABLED	não	false	UI OpenAPI
SENTRY_DSN	não	vazio	Telemetria
SENTRY_ENVIRONMENT	não	development	Tag ambiente
SENTRY_ENABLE_TRACING	não	false	Tracing Sentry
SENTRY_TRACES_SAMPLE_RATE	não	0.2	Amostragem
SERVICE_NAME	não	espaco-terapia-api	Campo log service
SERVICE_VERSION	não	v0.0.0	Campo log version
LOG_LEVEL	não	auto por env	debug/info/warn/error
LOG_FORMAT	não	auto	json text
LOG_INCLUDE_CALLER	não	false	AddSource slog
LOG_MASK_IP	não	true em prod	Mascara IP em access log
LOGIN_MAX_ATTEMPTS	não	5	Lockout progressivo
LOGIN_LOCKOUT_MINUTES	não	15	Duração bloqueio
RESEND_API_KEY	se e-mail	—	API Resend
EMAIL_FROM	se e-mail	—	Remetente
FRONTEND_PUBLIC_URL	não	localhost:5173	Links reset senha

Validação (cfg.Validate()):

- JWT_SECRET obrigatório se APP_ENV=production
- BOOTSTRAP_AUTH_ENABLED deve ser false em produção
- Token bootstrap obrigatório se bootstrap habilitado

2. Variáveis frontend (Vite)

Fonte: [.env.example](#).

Variável	Descrição
VITE_API_BASE_URL	Base API (/api/v1 com proxy dev)
VITE_API_*	Flags por módulo (ver cap. 01)

Variável	Descrição
VITE_AUTH_LOGIN	Login real vs desabilitado
VITE_AUTH_BOOTSTRAP	Bootstrap dev no browser

Segurança: nunca prefixar segredos com VITE_ (expostos no bundle). BOOTSTRAP_AUTH_TOKEN só no proxy Vite / servidor.

3. Matriz de ambientes

Aspecto	Development	Production
APP_ENV	development	production
Gin mode	Debug	ReleaseMode
Swagger	frequentemente true	false
Bootstrap auth	permitido	proibido
CORS	localhost	domínios reais
Logs	text ou json debug	json info+
Uploads	volume local	volume com permissões deploy/scripts/fix-uploads-permissions.sh

4. Docker Compose (local)

- make up — sobe api + db
- make migrate-up — profile ops, container migrate
- Health: GET http://localhost:8080/api/v1/health

Ver [DEV.md](#) e [DEPLOY.md](#) para procedimentos completos.

05 — Logging, correlação e observabilidade

1. Stack de logging

- **Biblioteca:** Go 1.21+ log/slog (structured logging)

- **Implementação:** [backend/internal/platform/logger/](#)
- **Adapter HTTP:** [backend/internal/infrastructure/logging](#) → `applog.New(cfg)`
- **Middleware Gin:** [middleware.AccessLog](#) → `logger.GinMiddleware`

2. Configuração

Env	Comportamento
LOG_LEVEL	debug, info, warn, error; se vazio, deriva de APP_ENV
LOG_FORMAT	json (prod) ou text (dev); vazio = auto
LOG_INCLUDE_CALLER	Adiciona arquivo:linha (AddSource)
LOG_MASK_IP	Substitui IP por [REDACTED] no access log

Campos default em todo log:

```
{
  "service": "espaco-terapia-api",
  "env": "production",
  "version": "v0.0.0",
  "level": "INFO",
  "msg": "..."
```

3. Correlação de requisições

Headers suportados:

Header	Constante	Uso
X-Request-ID	RequestIDHeader	ID único por request; gerado se ausente
X-Trace-Id	TraceIDHeader	Trace distribuído opcional
X-Tenant-Id	TenantIDHeader	Escopo multi-tenant futuro

O middleware:

1. Lê ou gera `request_id` (UUID)
2. Propaga no `context.Context`
3. Devolve no response header `X-Request-ID`
4. Sentry recebe tag `request_id` e `user_id` quando autenticado

4. Access log (HTTP)

Atributos típicos (sem body):

```
{
  "method": "GET",
  "path": "/api/v1/consultas",
  "status": 200,
  "latency_ms": 45,
  "request_id": "550e8400-e29b-41d4-a716-446655440000",
  "user_id": "..."}
}
```

Paths silenciosos (quiet): /api/v1/health, /api/v1/admin/health, /metrics, /api/v1/swagger/* (se IncludeSwagger=false).

Nunca logado: corpo request/response, headers Authorization, Cookie, Set-Cookie, X-API-Key, X-Bootstrap-Token.

5. Redação (sanitizer)

Arquivo: [sanitizer.go](https://github.com/elastic/sanitizer).

- RedactAttr aplicado em todos os handlers slog
- Denylist de chaves: password, token, cpf, jwt, database_url, prontuario, etc.
- JWT detectado por heurística → [REDACTED]
- Query params sensíveis redigidos em paths logados (SanitizePath)

6. Sentry

Inicialização em observability.InitSentry(cfg):

- SENTRY_DSN vazio = desabilitado
- Middleware sentrygin com Repanic: true
- Scope: request_id, user id do JWT
- Sample rate: SENTRY_TRACES_SAMPLE_RATE

7. Exemplos de eventos (referência)

Login falho (sem senha no log):

```
{"level":"WARN","msg":"login failed","request_id":"...","code":"INVALID_CREDENTIALS"}
```

Erro de validação 400:

```
{"level":"INFO","msg":"request completed","status":400,"path":"/api/v1/consultas","latency_ms":12}
```

Panic recovery: via `gin.Recovery()` + Sentry hub.

8. Boas práticas para novos módulos

1. Usar `logger.FromContext(ctx, base)` em services para propagar `request_id`
2. Não logar structs de paciente/prontuário completos
3. Erros ao cliente via `ErrorHandler` — stack apenas no servidor/Sentry

06 — API REST e documentação Swagger

1. Convenções gerais

Item	Valor
Base URL	/api/v1
Autenticação	Authorization: Bearer <access_token>
Content-Type	application/json (exceto uploads multipart/form-data)
OpenAPI	Swagger 2.0
Título	Espaço Terapia API v1.0

Anotações na raiz: [backend/cmd/api/main.go](#).

2. Habilitar Swagger UI

```
# backend/.env
SWAGGER_ENABLED=true
```

URLs:

- UI: <http://localhost:8080/api/v1/swagger/index.html>
- JSON: <http://localhost:8080/api/v1/swagger/doc.json>

Registro condicional em [routes/routes.go](#):

```
if cfg.SwaggerEnabled {
    api.GET("/swagger/*any", ginSwagger.WrapHandler(swaggerFiles.Handler))
}
```

Produção: manter SWAGGER_ENABLED=false (superfície de ataque / vazamento de schema).

3. Regenerar documentação

```
cd backend
make swagger
```

- Entrada: anotações @Router, @Summary, @Security BearerAuth nos handlers
- Saída: backend/docs/docs.go, swagger.json, swagger.yaml
- Script auxiliar opcional: scripts/inject_swagger_annotations.py

4. Verificação automatizada

```
cd backend && make verify-routes
```

- Smoke de rotas críticas com JWT
- Conta paths em swagger/doc.json
- Avisa se handlers novos não têm @Router

5. Catálogo de tags (domínios)

Agrupamento lógico alinhado aos register_*.go:

Tag / domínio	Prefixos principais
Auth	/auth/*
Pacientes	/pacientes
Profissionais	/profissionais, documentos
Consultas	/consultas
Salas	/salas
Terapias	/terapias
Anamneses	/anamneses, /respostas-anamnese
Prontuário	/prontuario/*

Tag / domínio	Prefixos principais
Financeiro	/financeiro/*
Contabilidade	/contabilidade/*
RH	/rh/*
Estoque	/estoque/*
Comodatos	/comodatos
Planos	/planos-saude, /acoes-judiciais, /notas-fiscais
Contratos	/contratos + rotas públicas
Marketing	/marketing/*
Documentos	/documentos/*
Chave digital	/chave-digital, /documentos-assinados
Access control	/access-control/*
Users	/users
Audit	/audit-log

Consulte `swagger/doc.json` para lista exata de paths (~centenas após Wave 3).

6. Contrato de resposta

Documentado em handlers com `@Success 200 {object} ....` Padrão runtime:

Sucesso: { "data": T, "meta": M }

Erro: { "error": { "code", "message", "details" } }

Códigos comuns: `VALIDATION_ERROR`, `NOT_FOUND`, `FORBIDDEN`, `UNAUTHORIZED`, `CONFLICT`, `INTERNAL_ERROR`.

7. Endpoints públicos (sem JWT)

Método	Path	Uso
POST	/auth/login	Credenciais
POST	/auth/forgot-password	Reset e-mail
POST	/auth/reset-password	Token reset

Método	Path	Uso
POST	/auth/token	Bootstrap dev
GET	/contratos/compartilhado/:token	Visualização pública
POST	/contratos/assinatura/:token/aceitar	Assinatura pública

8. Paginação (listagens)

Query params típicos (quando implementado no handler):

- page, per_page ou limit, offset
- Filtros: unidade_id, status, q (busca)

meta retorna totais para UI paginada.

07 — Autenticação, autorização e RBAC

1. Modelo de autenticação

- **Esquema:** JWT stateless (access token) + tabela jwt_revocations para logout/revogação
- **Emissor:** JWT_ISSUER (default espaco-terapia-api)
- **Transporte:** header Authorization: Bearer <token>

2. Fluxo de login

Diagrama (ver Markdown): sequenceDiagram...

Endpoints:

Método	Path	Auth
POST	/auth/login	Não
POST	/auth/logout	Sim — revoga JWT
GET	/auth/me	Sim
PATCH	/auth/me	Sim
PUT	/auth/me/password	Sim
POST	/auth/forgot-password	Não

Método	Path	Auth
POST	/auth/reset-password	Não
POST	/auth/token	Bootstrap (X-Bootstrap-Token) — só dev

Migrations relevantes: 000003_core_auth, 000017_auth_security, 000019_jwt_revocations, 000020_password_reset.

3. Frontend — AuthContext

Arquivo: [src/contexts/AuthContext.tsx](#).

- Persiste token via [tokenStore](#)
- hasPermission(permission: string) — conjunto de permissões do /auth/me
- ProtectedRoute redireciona para /login se não autenticado
- mustChangePassword → /alterar-senha

Cliente HTTP: [src/lib/api/client.ts](#) — injeta Bearer; 401 dispara logout global.

4. RBAC — permissões e roles

Migrations: 000021_rbac_permissions, 000022_user_roles, 000023_rbac_data_scopes.

Permissões de menu: strings menu.*.view definidas no seed RBAC; sidebar filtra por hasPermission.

Permissões de recurso: {recurso}.read|write|delete — guards nos register_*.go.

Gestão: página Controles de acesso (/configuracoes/controles-acesso):

Método	Path
GET	/access-control/permissions
GET	/access-control/data-scopes
GET	/access-control/roles/:role
PUT	/access-control/roles/:role

5. Escopo de dados (data scope)

DataScopeRepository restringe listagens por:

- Unidade do usuário

- Pacientes vinculados ao terapeuta (terapeuta_responsavel)
- Regras admin (acesso global)

Aplicado em services de pacientes, consultas, prontuário conforme role.

6. Lockout de login

Config: LOGIN_MAX_ATTEMPTS, LOGIN_LOCKOUT_MINUTES.

Tabela de tentativas (migration 000017): incremento em falha; bloqueio temporário.

7. Diagrama — request autenticado

Diagrama (ver Markdown): sequenceDiagram...

8. Segurança

- Produção: JWT_SECRET forte (≥ 32 bytes aleatórios)
- BOOTSTRAP_AUTH_ENABLED=false em produção
- Revogação no logout impede reuso de token roubado (até expiração, se não revogado)
- IDOR: sempre filtrar por user_id / unidade_id no service, não confiar em IDs do cliente

08 — Banco de dados PostgreSQL

1. Visão geral

- **SGBD:** PostgreSQL 16 (Docker)
- **Extensão:** pgcrypto (UUID)
- **Migrations:** 32 versões (000001–000032)
- **ORM:** GORM com modelos em infrastructure/database/*_model.go

2. Convenções de schema

Convenção	Detalhe
PK	UUID DEFAULT gen_random_uuid()
Timestamps	created_at, updated_at TIMESTAMPTZ
Soft delete	deleted_at onde aplicável

Convenção	Detalhe
Enums	CREATE TYPE ... AS ENUM
updated_at	trigger fn_set_updated_at()

3. ER — núcleo operacional

Diagrama (ver Markdown): erDiagram...

4. ER — clínico

Diagrama (ver Markdown): erDiagram...

5. ER — financeiro e planos

Diagrama (ver Markdown): erDiagram...

6. ER — RH, estoque, documentos

Diagrama (ver Markdown): erDiagram...

7. Índice de migrations

Migration	Domínio
000001	init
000002	pacientes, unidades
000003	auth users
000004	profissionais
000005	agenda, consultas, salas
000006	tratamentos
000007	prontuário
000008	anamneses
000009	financeiro
000010	planos
000011	RH

Migration	Domínio
000012	estoque
000013	contratos
000014	marketing
000015	contábil
000016–000020	auxiliar, audit, jwt, password reset
000021–000023	RBAC, data scopes
000024	rename terapias
000025–000027	paciente_profissionais, salas seed
000026	consultas.sala_id
000028	biblioteca documentos
000029	chaves digitais
000030–000031	contratos evolution/arquivo
000032	conciliação NF

8. Operações

```
cd backend && make migrate-up      # aplica UP
make migrate-down                 # reverte 1 versão
```

Produção: sempre backup antes de migration; nunca editar migration já aplicada.

9. Consultas críticas (agenda)

Tabela consultas — índices em paciente_id, profissional_id, data_hora, unidade_id (migration 000005/000026).

Enum consulta_status: agendada, confirmada, cancelada, concluida.

Enum status_atendimento: fluxo prontuário → aprovação.

09 — Arquitetura frontend (transversal)

1. Bootstrap da aplicação

Diagrama (ver Markdown): flowchart TD...

2. Layout e navegação

- **Sidebar:** [AppSidebar.tsx](#) — MENU_GROUPS por fluxo clínico
- **Header:** seletor de unidade, usuário, logout
- **Badges:** [useMenuBadges](#) — pendências consultas/profissionais

3. Cliente HTTP

[src/lib/api/client.ts](#):

- `apiRequest<T>(path, options) → { data, meta }`
- Timeout: `API_TIMEOUT_MS`
- Erros → `ApiClientError` com `code`, `details[]`
- Base URL: `getApiBaseUrl()` — proxy Vite em dev

4. React Query — padrões

Padrão	Exemplo
Query key	<code>['consultas', unidadeApiId, filters]</code>
Após mutation	<code>queryClient.refetchQueries({ queryKey: [...], type: 'active' })</code>
Unidade switch	UnidadeContext invalida consultas
401	sem retry; logout

5. UnidadeContext

[src/contexts/UnidadeContext.tsx](#):

- Lista unidades (API ou seed local)
- Persiste seleção (`localStorage`)
- Expõe `unidadeAtiva`, `unidadeApiId` (UUID)
- Resolver: [src/lib/unidades/apiIds.ts](#)

6. Matriz persistência (resumo)

Módulo	API	Fallback local
Pacientes	VITE_API_PACIENTES	localStorage legado
Consultas/Agenda	VITE_API_CONSULTAS	—
Profissionais	VITE_API_PROFISSIONAIS	parcial
Exceções agenda prof.	—	localStorage (/profissionais/:id/agenda)

7. Proxy Vite (dev)

vite.config.ts: /api → http://localhost:8080 — evita CORS em desenvolvimento.

8. Build produção

```
npm run build
# dist/ servido por Nginx; VITE_* embutidos no bundle no build-time
```

Importante: alterar flags API exige **rebuild** do frontend em produção.

9. Rotas não listadas no sidebar

Rota	Página
/	Dashboard
/prontuario/:consultaId	Prontuário sessão
/configuracoes/*	Configurações admin
/conta/*	Perfil/senha
/dashboard-ocupacao	Ocupação salas
/unidades	Unidades (menu oculto)
/contratos/compartilhado/:token	Público

Ver apêndice [rotas-auxiliares.md](#).

10 — DevOps, segurança e qualidade

1. Pipeline de deploy (resumo)

Fluxo documentado em [DEPLOY.md](#):

1. Build imagem API (multi-stage, usuário non-root)
2. Build SPA (npm run build) com VITE_* de produção
3. migrate-up antes ou durante deploy
4. Nginx TLS + proxy /api/v1
5. Volume uploads com permissões corretas (deploy/scripts/fix-uploads-permissions.sh)

2. Hardening

Controle	Implementação
Container non-root	Dockerfile API
CORS restritivo	lista explícita, sem * com credentials
JWT secret	obrigatório prod
Bootstrap off prod	config.Validate()
Rate limit login	tentativas + logout
Headers segurança	Nginx (ver SECURITY.md)
Swagger off prod	SWAGGER_ENABLED=false
Logs sem PII	sanitizer slog
Upload path	fora de webroot; download via handler autenticado

3. OWASP — mapeamento

Risco	Mitigação no projeto
Broken Auth	JWT + revogação + logout
Broken Access Control	RBAC + data scope + guards
IDOR	filtros no service por unidade/user
Injection	GORM parametrizado, validação DTO
Security misconfiguration	Validate config prod

Risco	Mitigação no projeto
Sensitive data exposure	redação logs, erros sem stack

4. Quality gates

```
# Backend
cd backend && go test ./... && go vet ./... && make verify-routes

# Frontend
npm run test && npm run lint

# E2E (agenda)
npm run test:e2e -- e2e/agenda-sync.spec.ts --project=agenda-sync
```

5. Observabilidade em produção

- Logs JSON para agregador (stdout Docker)
- Sentry para erros 5xx e panics
- Healthcheck: /api/v1/health
- Correlação: propagar X-Request-ID do load balancer

6. Débitos técnicos conhecidos

Item	Impacto	Referência
datetime-local enviado como UTC no backend	Validação expediente em BRT	DEV.md checklist agenda
/profissionais/:id/agenda não é lista de consultas	Confusão operacional	cap. Profissionais
Alguns módulos ainda híbridos localStorage	Dados divergentes sem API	featureFlags

Dashboard

1. Identificação

Campo	Valor
Menu	(raiz — fora do sidebar)
Rota SPA	/
Permissão RBAC	(autenticado)
Feature flag	–
Página	src/pages/Dashboard.tsx

2. UI e componentes

- Entrada principal: src/pages/Dashboard.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	vários use* conforme cards
API client	src/lib/api/ (módulo homônimo)
Feature flag	–

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/–
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: Agregações locais + endpoints pontuais conforme widgets

5. Persistência

Tabelas: consultas, pacientes (leitura)

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Agenda

1. Identificação

Campo	Valor
Menu	Recepção → Agenda
Rota SPA	/agenda
Permissão RBAC	menu.agenda.view

Campo	Valor
Feature flag	VITE_API_CONSULTAS
Página	src/pages/Agenda.tsx

2. UI e componentes

- Entrada principal: src/pages/Agenda.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	useConsultas
API client	src/lib/api/ (módulo homônimo)
Feature flag	VITE_API_CONSULTAS

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_consultas.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: GET /consultas?unidade_id=&de=&ate=

5. Persistência

Tabelas: consultas, salas, pacientes, profissionais

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): `flowchart LR...`

7. Sequência — leitura lista

Diagrama (ver Markdown): `sequenceDiagram...`

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Agendamentos

1. Identificação

Campo	Valor
Menu	Recepção → Agendamentos
Rota SPA	<code>/consultas</code>
Permissão RBAC	<code>menu.consultas.view</code>
Feature flag	<code>VITE_API_CONSULTAS</code>
Página	<code>src/pages/Consultas.tsx</code>

2. UI e componentes

- Entrada principal: `src/pages/Consultas.tsx`
- Layout: `AppLayout` + sidebar filtrada por permissão

- Formulários/modais: ver subpasta `src/components/` do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	<code>useConsultas</code>
API client	<code>src/lib/api/</code> (módulo homônimo)
Feature flag	<code>VITE_API_CONSULTAS</code>

Modo de dados: API quando flag `true`; caso contrário fallback `localStorage` (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	<code>backend/internal/interfaces/http/routes/register_consultas.go</code>
Handlers	<code>backend/internal/interfaces/http/handlers/</code>
Services	<code>backend/internal/domain/service/</code>

Endpoints principais: CRUD `/consultas` + POST `confirmar|cancelar|concluir|vincular-prontuario|aprovar|rejeitar`

5. Persistência

Tabelas: consultas

Consultar migrations em `backend/migrations/` e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): `flowchart LR...`

7. Sequência — leitura lista

Diagrama (ver Markdown): `sequenceDiagram...`

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Endpoints detalhados (`/api/v1/consultas`)

Método	Path	Ação
GET	<code>/consultas</code>	Lista (filtros: <code>unidade_id</code> , <code>datas</code> , <code>status</code>)
GET	<code>/consultas/:id</code>	Detalhe
POST	<code>/consultas</code>	Criar
PUT	<code>/consultas/:id</code>	Atualizar
DELETE	<code>/consultas/:id</code>	Remover
POST	<code>/consultas/:id/confirmar</code>	Confirmar presença
POST	<code>/consultas/:id/cancelar</code>	Cancelar
POST	<code>/consultas/:id/concluir</code>	Concluir atendimento
POST	<code>/consultas/:id/vincular-prontuario</code>	Associar evolução
POST	<code>/consultas/:id/aprovar-atendimento</code>	Aprovação gestor
POST	<code>/consultas/:id/rejeitar-atendimento</code>	Rejeição com motivo

10. Sincronização com Agenda (`/agenda`)

- `useConsultas` usa `unidade.apiId` (UUID) — ver `resolveUnidadeApiIdFromContext`
- Após mutação: `refetchQueries` em chaves `['consultas', unidadeApiId]`
- `Consultas.tsx`: `submit` **await** + `isSubmitting` — modal só fecha em sucesso

11. Débito técnico — fuso horário

O backend interpreta `datetime-local` do HTML como **UTC** (`ParseDateTime`). Em BRT, horários próximos à meia-noite ou validação de expediente podem falhar com 400. Workaround E2E: usar horário 13:00 UTC. Correção definitiva: normalizar para `America/Sao_Paulo` no handler.

12. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- sala_id obrigatório (migration 000026) — conflito de sala retorna 409

Pacientes

1. Identificação

Campo	Valor
Menu	Recepção → Pacientes
Rota SPA	/pacientes
Permissão RBAC	menu.pacientes.view
Feature flag	VITE_API_PACIENTES
Página	src/pages/Pacientes.tsx

2. UI e componentes

- Entrada principal: src/pages/Pacientes.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	usePacientes
API client	src/lib/api/ (módulo homônimo)
Feature flag	VITE_API_PACIENTES

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_pacientes.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: CRUD /pacientes + POST /:id/restore

5. Persistência

Tabelas: pacientes, paciente_unidades, paciente_profissionais

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
 - Aplicar data scope por unidade/usuário em services de listagem
 - Não expor IDs de outros tenants/unidades na resposta
-

Terapias

1. Identificação

Campo	Valor
Menu	Recepção → Terapias
Rota SPA	/terapias
Permissão RBAC	menu.terapias.view
Feature flag	VITE_API_TERAPIAS
Página	src/pages/Terapias.tsx

2. UI e componentes

- Entrada principal: src/pages/Terapias.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	useTerapias
API client	src/lib/api/ (módulo homônimo)
Feature flag	VITE_API_TERAPIAS

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_terapias.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: CRUD /terapias

5. Persistência

Tabelas: terapias (ex-tratamentos, mig. 000024)

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Salas de Atendimento

1. Identificação

Campo	Valor
Menu	Recepção → Salas
Rota SPA	/salas
Permissão RBAC	menu.salas.view

Campo	Valor
Feature flag	VITE_API_SALAS
Página	src/pages/Salas.tsx, AgendaSala.tsx

2. UI e componentes

- Entrada principal: src/pages/Salas.tsx, AgendaSala.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	useSalas
API client	src/lib/api/ (módulo homônimo)
Feature flag	VITE_API_SALAS

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_salas.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: CRUD /salas

5. Persistência

Tabelas: salas, sala_especialidades, sala_recursos

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): `flowchart LR...`

7. Sequência — leitura lista

Diagrama (ver Markdown): `sequenceDiagram...`

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Prontuários

1. Identificação

Campo	Valor
Menu	Clínico → Prontuários
Rota SPA	<code>/prontuarios</code> , <code>/prontuario/:consultaId</code>
Permissão RBAC	<code>menu.prontuarios.view</code>
Feature flag	<code>VITE_API_PRONTUARIO</code>
Página	<code>src/pages/Prontuarios.tsx</code> , <code>Prontuario.tsx</code>

2. UI e componentes

- Entrada principal: `src/pages/Prontuarios.tsx`, `Prontuario.tsx`
- Layout: `AppLayout` + sidebar filtrada por permissão

- Formulários/modais: ver subpasta `src/components/` do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	<code>useProntuario</code>
API client	<code>src/lib/api/</code> (módulo homônimo)
Feature flag	<code>VITE_API_PRONTUARIO</code>

Modo de dados: API quando flag `true`; caso contrário fallback `localStorage` (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	<code>backend/internal/interfaces/http/routes/register_prontuario.go</code>
Handlers	<code>backend/internal/interfaces/http/handlers/</code>
Services	<code>backend/internal/domain/service/</code>

Endpoints principais: GET `/prontuario/pacientes/:id`; POST `evolucoes|prescricoes|atestados|documentos`

5. Persistência

Tabelas: `prontuarios`, `prontuario_*`

Consultar migrations em `backend/migrations/` e ER em [08-banco-dados.md](#).

6. Fluxo operacional

| Diagrama (ver Markdown): `flowchart LR...`

7. Sequência — leitura lista

| Diagrama (ver Markdown): `sequenceDiagram...`

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Anamneses

1. Identificação

Campo	Valor
Menu	Clínico → Anamneses
Rota SPA	<code>/anamneses</code>
Permissão RBAC	<code>menu.anamneses.view</code>
Feature flag	<code>VITE_API_ANAMNESES</code>
Página	<code>src/pages/Anamneses.tsx</code>

2. UI e componentes

- Entrada principal: `src/pages/Anamneses.tsx`
- Layout: `AppLayout` + sidebar filtrada por permissão
- Formulários/modais: ver subpasta `src/components/` do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	<code>useAnamneses</code>
API client	<code>src/lib/api/</code> (módulo homônimo)

Artefato	Caminho
Feature flag	VITE_API_ANAMNESES

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_anamneses.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: CRUD /anamneses; /respostas-anamnese

5. Persistência

Tabelas: anamneses, respostas_anamnese

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem

- Não expor IDs de outros tenants/unidades na resposta

Aprovação de Atendimentos

1. Identificação

Campo	Valor
Menu	Clínico → Aprovação Atendimentos
Rota SPA	/atendimentos/aprovacoes
Permissão RBAC	menu.aprovacoes.view
Feature flag	VITE_API_CONSULTAS
Página	src/pages/AtendimentosAprovacao.tsx

2. UI e componentes

- Entrada principal: src/pages/AtendimentosAprovacao.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	useConsultas
API client	src/lib/api/ (módulo homônimo)
Feature flag	VITE_API_CONSULTAS

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_consultas.go

Artefato	Caminho
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: POST /consultas/:id/aprovar-atendimento|rejeitar-atendimento

5. Persistência

Tabelas: consultas.status_atendimento

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
 - Aplicar data scope por unidade/usuário em services de listagem
 - Não expor IDs de outros tenants/unidades na resposta
-

Documentos Assinados

1. Identificação

Campo	Valor
Menu	Clínico → Docs Assinados
Rota SPA	/documentos-assinados
Permissão RBAC	menu.docs-assinados.view
Feature flag	VITE_API_DOCUMENTOS_ASSINADOS
Página	src/pages/DocumentosAssinados.tsx

2. UI e componentes

- Entrada principal: `src/pages/DocumentosAssinados.tsx`
- Layout: `AppLayout` + sidebar filtrada por permissão
- Formulários/modais: ver subpasta `src/components/` do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	<code>useDocumentosAssinados</code>
API client	<code>src/lib/api/</code> (módulo homônimo)
Feature flag	<code>VITE_API_DOCUMENTOS_ASSINADOS</code>

Modo de dados: API quando flag `true`; caso contrário fallback `localStorage` (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	<code>backend/internal/interfaces/http/routes/register_chave_digital.go</code>
Handlers	<code>backend/internal/interfaces/http/handlers/</code>
Services	<code>backend/internal/domain/service/</code>

Endpoints principais: GET /documentos-assinados; POST assinar; GET download; POST verificar

5. Persistência

Tabelas: documentos_assinados, chaves_digitais

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Meu Painel

1. Identificação

Campo	Valor
Menu	Profissionais → Meu Painel
Rota SPA	/meu-painel
Permissão RBAC	menu.meu-painel.view

Campo	Valor
Feature flag	VITE_API_CONSULTAS, VITE_API_PROFISSIONAIS
Página	src/pages/MeuPainel.tsx

2. UI e componentes

- Entrada principal: `src/pages/MeuPainel.tsx`
- Layout: `AppLayout` + sidebar filtrada por permissão
- Formulários/modais: ver subpasta `src/components/` do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	<code>useConsultas</code> , <code>useAuth</code>
API client	<code>src/lib/api/</code> (módulo homônimo)
Feature flag	VITE_API_CONSULTAS, VITE_API_PROFISSIONAIS

Modo de dados: API quando flag `true`; caso contrário fallback `localStorage` (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	<code>backend/internal/interfaces/http/routes/vários</code>
Handlers	<code>backend/internal/interfaces/http/handlers/</code>
Services	<code>backend/internal/domain/service/</code>

Endpoints principais: Métricas do profissional logado

5. Persistência

Tabelas: consultas, profissionais

Consultar migrations em `backend/migrations/` e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): `flowchart LR...`

7. Sequência — leitura lista

Diagrama (ver Markdown): `sequenceDiagram...`

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Minha Agenda

1. Identificação

Campo	Valor
Menu	Profissionais → Minha Agenda
Rota SPA	<code>/minha-agenda</code>
Permissão RBAC	<code>menu.minha-agenda.view</code>
Feature flag	<code>VITE_API_CONSULTAS</code>
Página	<code>src/pages/MinhaAgenda.tsx</code>

2. UI e componentes

- Entrada principal: `src/pages/MinhaAgenda.tsx`
- Layout: `AppLayout` + sidebar filtrada por permissão

- Formulários/modais: ver subpasta `src/components/` do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	<code>useConsultas</code>
API client	<code>src/lib/api/</code> (módulo homônimo)
Feature flag	<code>VITE_API_CONSULTAS</code>

Modo de dados: API quando flag `true`; caso contrário fallback `localStorage` (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	<code>backend/internal/interfaces/http/routes/register_consultas.go</code>
Handlers	<code>backend/internal/interfaces/http/handlers/</code>
Services	<code>backend/internal/domain/service/</code>

Endpoints principais: `GET /consultas` filtrado `profissional_id=me`

5. Persistência

Tabelas: consultas

Consultar migrations em `backend/migrations/` e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): `flowchart LR...`

7. Sequência — leitura lista

Diagrama (ver Markdown): `sequenceDiagram...`

8. Testes

- Backend: `go test ./internal/domain/service/...`

- Frontend: *.test.tsx no domínio, se existir
- E2E: ver e2e/ para fluxos críticos (ex. agenda em agenda-sync.spec.ts)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
 - Aplicar data scope por unidade/usuário em services de listagem
 - Não expor IDs de outros tenants/unidades na resposta
-

Profissionais

1. Identificação

Campo	Valor
Menu	Profissionais → Profissionais
Rota SPA	/profissionais, /profissionais/:id/agenda
Permissão RBAC	menu.profissionais.view
Feature flag	VITE_API_PROFISSIONAIS
Página	src/pages/Profissionais.tsx, AgendaProfissional.tsx

2. UI e componentes

- Entrada principal: src/pages/Profissionais.tsx, AgendaProfissional.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	useProfissionais
API client	src/lib/api/ (módulo homônimo)
Feature flag	VITE_API_PROFISSIONAIS

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_profissionais.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: CRUD /profissionais; documentos upload; /profissionais/documentos/pendencias

5. Persistência

Tabelas: profissionais, profissional_documentos, conselhos

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: go test ./internal/domain/service/...
- Frontend: *.test.tsx no domínio, se existir
- E2E: ver e2e/ para fluxos críticos (ex. agenda em agenda-sync.spec.ts)

9. Distinção crítica: /profissionais/:id/agenda

Rota	Fonte de dados	Propósito
/agenda, /consultas, /minha-agenda	API GET /consultas	Consultas reais

Rota	Fonte de dados	Propósito
/profissionais/:id/agenda	localStorage (AgendaProfissional.tsx)	Exceções de disponibilidade (férias, almoço)

Não confundir: alterações na agenda de exceções **não** aparecem na agenda de consultas.

10. Documentos do profissional

Método	Path
GET	/profissionais/documentos/pendencias
GET	/profissionais/:id/documentos
POST	/profissionais/:id/documentos (multipart)
GET	/profissionais/:id/documentos/:docId/download
DELETE	/profissionais/:id/documentos/:docId

11. Segurança

- Validar RBAC no guard HTTP antes do handler
- Upload: validar MIME/tamanho no service; path fora de webroot
- Pendências expostas no badge sidebar profissionais_pendentes

Financeiro

1. Identificação

Campo	Valor
Menu	Financeiro → Financeiro
Rota SPA	/financeiro
Permissão RBAC	menu.financeiro.view
Feature flag	VITE_API_FINANCEIRO
Página	src/pages/Financeiro.tsx

2. UI e componentes

- Entrada principal: `src/pages/Financeiro.tsx`
- Layout: `AppLayout` + sidebar filtrada por permissão
- Formulários/modais: ver subpasta `src/components/` do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	<code>useFinanceiro</code>
API client	<code>src/lib/api/</code> (módulo homônimo)
Feature flag	<code>VITE_API_FINANCEIRO</code>

Modo de dados: API quando flag `true`; caso contrário fallback `localStorage` (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	<code>backend/internal/interfaces/http/routes/register_financeiro.go</code>
Handlers	<code>backend/internal/interfaces/http/handlers/</code>
Services	<code>backend/internal/domain/service/</code>

Endpoints principais: `/financeiro/categorias|centros-custo|lancamentos`

5. Persistência

Tabelas: `financeiro_*`

Consultar migrations em `backend/migrations/` e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): `flowchart LR...`

7. Sequência — leitura lista

Diagrama (ver Markdown): `sequenceDiagram...`

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Balancetes

1. Identificação

Campo	Valor
Menu	Financeiro → Balancetes
Rota SPA	/balancetes
Permissão RBAC	menu.balancetes.view
Feature flag	VITE_API_CONTABILIDADE
Página	src/pages/Balancetes.tsx

2. UI e componentes

- Entrada principal: `src/pages/Balancetes.tsx`
- Layout: `AppLayout` + sidebar filtrada por permissão
- Formulários/modais: ver subpasta `src/components/` do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	<code>useContabilidade</code>
API client	<code>src/lib/api/</code> (módulo homônimo)

Artefato	Caminho
Feature flag	VITE_API_CONTABILIDADE

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_contabilidade.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: GET /contabilidade/balancete; CRUD contas e lançamentos contabeis

5. Persistência

Tabelas: contas_contabeis, lancamentos_contabeis

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem

- Não expor IDs de outros tenants/unidades na resposta

Relatórios de Conciliação

1. Identificação

Campo	Valor
Menu	Financeiro → Relatórios Conciliação
Rota SPA	/relatorios-conciliacao
Permissão RBAC	menu.relatorios-conciliacao.view
Feature flag	VITE_API_PLANOS
Página	src/pages/RelatoriosConciliacao.tsx

2. UI e componentes

- Entrada principal: src/pages/RelatoriosConciliacao.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	usePlanos
API client	src/lib/api/ (módulo homônimo)
Feature flag	VITE_API_PLANOS

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_planos.go

Artefato	Caminho
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: GET /acoes-judiciais/conciliacao-resumo

5. Persistência

Tabelas: acoes_judiciais, notas_fiscais

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
 - Aplicar data scope por unidade/usuário em services de listagem
 - Não expor IDs de outros tenants/unidades na resposta
-

Auditoria de Notas

1. Identificação

Campo	Valor
Menu	Financeiro → Auditoria de Notas
Rota SPA	/auditoria-notas
Permissão RBAC	menu.auditoria-notas.view
Feature flag	VITE_API_PLANOS
Página	src/pages/AuditoriaNotas.tsx

2. UI e componentes

- Entrada principal: `src/pages/AuditoriaNotas.tsx`
- Layout: `AppLayout` + sidebar filtrada por permissão
- Formulários/modais: ver subpasta `src/components/` do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	<code>useNotasFiscais</code>
API client	<code>src/lib/api/</code> (módulo homônimo)
Feature flag	<code>VITE_API_PLANOS</code>

Modo de dados: API quando flag `true`; caso contrário fallback `localStorage` (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	<code>backend/internal/interfaces/http/routes/register_planos.go</code>
Handlers	<code>backend/internal/interfaces/http/handlers/</code>
Services	<code>backend/internal/domain/service/</code>

Endpoints principais: CRUD /notas-fiscais; POST /:id/conciliar

5. Persistência

Tabelas: notas_fiscais (mig. 000032)

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Contratos

1. Identificação

Campo	Valor
Menu	RH & Contratos → Contratos
Rota SPA	/contratos
Permissão RBAC	menu.contratos.view

Campo	Valor
Feature flag	VITE_API_CONTRATOS
Página	src/pages/Contratos.tsx

2. UI e componentes

- Entrada principal: src/pages/Contratos.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	useContratos
API client	src/lib/api/ (módulo homônimo)
Feature flag	VITE_API_CONTRATOS

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_contratos.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: CRUD /contratos; arquivo; compartilhar; solicitacoes-assinatura; rotas publicas token

5. Persistência

Tabelas: contratos, contrato_arquivos, solicitacoes

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): `flowchart LR...`

7. Sequência — leitura lista

Diagrama (ver Markdown): `sequenceDiagram...`

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Folha de Pagamento

1. Identificação

Campo	Valor
Menu	RH → Folha de Pagamento
Rota SPA	/folha-pagamento
Permissão RBAC	menu.folha-pagamento.view
Feature flag	VITE_API_RH
Página	src/pages/FolhaPagamento.tsx

2. UI e componentes

- Entrada principal: `src/pages/FolhaPagamento.tsx`
- Layout: `AppLayout` + sidebar filtrada por permissão

- Formulários/modais: ver subpasta `src/components/` do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	<code>useRH</code>
API client	<code>src/lib/api/</code> (módulo homônimo)
Feature flag	<code>VITE_API_RH</code>

Modo de dados: API quando flag `true`; caso contrário fallback `localStorage` (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	<code>backend/internal/interfaces/http/routes/register_rh.go</code>
Handlers	<code>backend/internal/interfaces/http/handlers/</code>
Services	<code>backend/internal/domain/service/</code>

Endpoints principais: `/rh/funcionarios-clt|pj`; `/rh/folhas-clt|pj`

5. Persistência

Tabelas: `funcionarios_`, `folhas_`

Consultar migrations em `backend/migrations/` e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): `flowchart LR...`

7. Sequência — leitura lista

Diagrama (ver Markdown): `sequenceDiagram...`

8. Testes

- Backend: `go test ./internal/domain/service/...`

- Frontend: *.test.tsx no domínio, se existir
- E2E: ver e2e/ para fluxos críticos (ex. agenda em agenda-sync.spec.ts)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
 - Aplicar data scope por unidade/usuário em services de listagem
 - Não expor IDs de outros tenants/unidades na resposta
-

Planos de Saúde

1. Identificação

Campo	Valor
Menu	Planos & Jurídico → Planos de Saúde
Rota SPA	/planos-saude
Permissão RBAC	menu.planos-saude.view
Feature flag	VITE_API_PLANOS
Página	src/pages/PlanosSaude.tsx

2. UI e componentes

- Entrada principal: src/pages/PlanosSaude.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	usePlanosSaude
API client	src/lib/api/ (módulo homônimo)
Feature flag	VITE_API_PLANOS

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_planos.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: CRUD /planos-saude

5. Persistência

Tabelas: planos_saude

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: go test ./internal/domain/service/...
- Frontend: *.test.tsx no domínio, se existir
- E2E: ver e2e/ para fluxos críticos (ex. agenda em agenda-sync.spec.ts)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
 - Aplicar data scope por unidade/usuário em services de listagem
 - Não expor IDs de outros tenants/unidades na resposta
-

Ações Judiciais

1. Identificação

Campo	Valor
Menu	Planos & Jurídico → Ações Judiciais
Rota SPA	/acoes-judiciais, /acoes-judiciais/:id
Permissão RBAC	menu.acoes-judiciais.view
Feature flag	VITE_API_PLANOS
Página	src/pages/AcoesJudiciais.tsx, AcaoJudicialDetalhe.tsx

2. UI e componentes

- Entrada principal: src/pages/AcoesJudiciais.tsx, AcaoJudicialDetalhe.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	useAcoesJudiciais
API client	src/lib/api/ (módulo homônimo)
Feature flag	VITE_API_PLANOS

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_planos.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: CRUD /acoes-judiciais; GET conciliacao

5. Persistência

Tabelas: acoes_judiciais

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Estoque

1. Identificação

Campo	Valor
Menu	Estoque & Ativos → Estoque
Rota SPA	/estoque
Permissão RBAC	menu.estoque.view

Campo	Valor
Feature flag	VITE_API_ESTOQUE
Página	src/pages/Estoque.tsx

2. UI e componentes

- Entrada principal: src/pages/Estoque.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	useEstoque
API client	src/lib/api/ (módulo homônimo)
Feature flag	VITE_API_ESTOQUE

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_estoque.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: /estoque/itens|movimentacoes|inventarios

5. Persistência

Tabelas: estoque_*

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): `flowchart LR...`

7. Sequência — leitura lista

Diagrama (ver Markdown): `sequenceDiagram...`

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Comodato

1. Identificação

Campo	Valor
Menu	Estoque → Comodato
Rota SPA	<code>/comodato</code>
Permissão RBAC	<code>menu.comodato.view</code>
Feature flag	<code>VITE_API_COMODATO</code>
Página	<code>src/pages/Comodato.tsx</code>

2. UI e componentes

- Entrada principal: `src/pages/Comodato.tsx`
- Layout: `AppLayout` + sidebar filtrada por permissão

- Formulários/modais: ver subpasta `src/components/` do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	<code>useComodatos</code>
API client	<code>src/lib/api/</code> (módulo homônimo)
Feature flag	<code>VITE_API_COMODATO</code>

Modo de dados: API quando flag `true`; caso contrário fallback `localStorage` (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	<code>backend/internal/interfaces/http/routes/register_comodatos.go</code>
Handlers	<code>backend/internal/interfaces/http/handlers/</code>
Services	<code>backend/internal/domain/service/</code>

Endpoints principais: CRUD `/comodatos`

5. Persistência

Tabelas: `comodatos`

Consultar migrations em `backend/migrations/` e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): `flowchart LR...`

7. Sequência — leitura lista

Diagrama (ver Markdown): `sequenceDiagram...`

8. Testes

- Backend: `go test ./internal/domain/service/...`

- Frontend: *.test.tsx no domínio, se existir
- E2E: ver e2e/ para fluxos críticos (ex. agenda em agenda-sync.spec.ts)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Relatórios

1. Identificação

Campo	Valor
Menu	Relatórios → Relatórios
Rota SPA	/relatorios
Permissão RBAC	menu.relatorios.view
Feature flag	VITE_API_RELATORIOS
Página	src/pages/Relatorios.tsx

2. UI e componentes

- Entrada principal: src/pages/Relatorios.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	useRelatorios
API client	src/lib/api/ (módulo homônimo)
Feature flag	VITE_API_RELATORIOS

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_relatorios_operacionais.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: CRUD /relatorios-operacionais

5. Persistência

Tabelas: relatorios_operacionais

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
 - Aplicar data scope por unidade/usuário em services de listagem
 - Não expor IDs de outros tenants/unidades na resposta
-

Relatórios Avançados

1. Identificação

Campo	Valor
Menu	Relatórios → Relatórios Avançados
Rota SPA	/relatorios-avancados
Permissão RBAC	menu.relatorios-avancados.view
Feature flag	VITE_API_RELATORIOS
Página	src/pages/RelatoriosAvancados.tsx

2. UI e componentes

- Entrada principal: `src/pages/RelatoriosAvancados.tsx`
- Layout: `AppLayout` + sidebar filtrada por permissão
- Formulários/modais: ver subpasta `src/components/` do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	<code>useRelatorios</code>
API client	<code>src/lib/api/</code> (módulo homônimo)
Feature flag	<code>VITE_API_RELATORIOS</code>

Modo de dados: API quando flag `true`; caso contrário fallback `localStorage` (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	<code>backend/internal/interfaces/http/routes/register_relatorios_operacionais.go</code>
Handlers	<code>backend/internal/interfaces/http/handlers/</code>
Services	<code>backend/internal/domain/service/</code>

Endpoints principais: Mesma API com filtros avançados / export

5. Persistência

Tabelas: relatorios_operacionais

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Marketing

1. Identificação

Campo	Valor
Menu	Marketing → Marketing
Rota SPA	/marketing
Permissão RBAC	menu.marketing.view

Campo	Valor
Feature flag	VITE_API_MARKETING
Página	src/pages/Marketing.tsx

2. UI e componentes

- Entrada principal: `src/pages/Marketing.tsx`
- Layout: `AppLayout` + sidebar filtrada por permissão
- Formulários/modais: ver subpasta `src/components/` do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	<code>useMarketing</code>
API client	<code>src/lib/api/</code> (módulo homônimo)
Feature flag	VITE_API_MARKETING

Modo de dados: API quando `flag true`; caso contrário fallback `localStorage` (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	<code>backend/internal/interfaces/http/routes/register_marketing.go</code>
Handlers	<code>backend/internal/interfaces/http/handlers/</code>
Services	<code>backend/internal/domain/service/</code>

Endpoints principais: `/marketing/manuais|materiais + upload/download`

5. Persistência

Tabelas: `marketing_manuais`, `marketing_materiais`

Consultar migrations em `backend/migrations/` e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): `flowchart LR...`

7. Sequência — leitura lista

Diagrama (ver Markdown): `sequenceDiagram...`

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Documentos (Biblioteca)

1. Identificação

Campo	Valor
Menu	Documentos → Documentos
Rota SPA	/documentos
Permissão RBAC	menu.documentos.view
Feature flag	VITE_API_DOCUMENTOS
Página	src/pages/Documentos.tsx

2. UI e componentes

- Entrada principal: `src/pages/Documentos.tsx`
- Layout: `AppLayout` + sidebar filtrada por permissão

- Formulários/modais: ver subpasta `src/components/` do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	<code>useBibliotecaDocumentos</code>
API client	<code>src/lib/api/</code> (módulo homônimo)
Feature flag	<code>VITE_API_DOCUMENTOS</code>

Modo de dados: API quando flag `true`; caso contrário fallback `localStorage` (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	<code>backend/internal/interfaces/http/routes/register_documentos.go</code>
Handlers	<code>backend/internal/interfaces/http/handlers/</code>
Services	<code>backend/internal/domain/service/</code>

Endpoints principais: `/documentos/categorias`; `/documentos/arquivos upload|download`

5. Persistência

Tabelas: `documento_categorias`, `biblioteca_arquivos` (mig. 000028)

Consultar migrations em `backend/migrations/` e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): `flowchart LR...`

7. Sequência — leitura lista

Diagrama (ver Markdown): `sequenceDiagram...`

8. Testes

- Backend: `go test ./internal/domain/service/...`

- Frontend: *.test.tsx no domínio, se existir
- E2E: ver e2e/ para fluxos críticos (ex. agenda em agenda-sync.spec.ts)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
 - Aplicar data scope por unidade/usuário em services de listagem
 - Não expor IDs de outros tenants/unidades na resposta
-

Usuários

1. Identificação

Campo	Valor
Menu	Administração → Usuários
Rota SPA	/configuracoes/usuarios
Permissão RBAC	menu.configuracoes.usuarios.view
Feature flag	–
Página	src/pages/Configuracoes.tsx (aba usuários)

2. UI e componentes

- Entrada principal: src/pages/Configuracoes.tsx (aba usuários)
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	useUsers
API client	src/lib/api/ (módulo homônimo)
Feature flag	–

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_users.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: CRUD /users + POST restore (admin)

5. Persistência

Tabelas: users, user_roles

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
 - Aplicar data scope por unidade/usuário em services de listagem
 - Não expor IDs de outros tenants/unidades na resposta
-

Controles de Acesso

1. Identificação

Campo	Valor
Menu	Administração → Controles de acesso
Rota SPA	/configuracoes/controles-acesso
Permissão RBAC	menu.configuracoes.acessos.view
Feature flag	–
Página	src/pages/ControlesAcesso.tsx

2. UI e componentes

- Entrada principal: src/pages/ControlesAcesso.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	useAccessControl
API client	src/lib/api/ (módulo homônimo)
Feature flag	–

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_access_control.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: /access-control/permissions|data-scopes|roles/:role

5. Persistência

Tabelas: permissions, role_permissions, data_scopes

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): flowchart LR...

7. Sequência — leitura lista

Diagrama (ver Markdown): sequenceDiagram...

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Chave Digital

1. Identificação

Campo	Valor
Menu	Administração → Chave Digital
Rota SPA	/configuracoes/chave-digital
Permissão RBAC	menu.configuracoes.chave-digital.view

Campo	Valor
Feature flag	VITE_API_CHAVE_DIGITAL
Página	src/pages/ChaveDigital.tsx

2. UI e componentes

- Entrada principal: src/pages/ChaveDigital.tsx
- Layout: AppLayout + sidebar filtrada por permissão
- Formulários/modais: ver subpasta src/components/ do domínio

3. Integração frontend

Artefato	Caminho
Hook(s)	useChaveDigital
API client	src/lib/api/ (módulo homônimo)
Feature flag	VITE_API_CHAVE_DIGITAL

Modo de dados: API quando flag true; caso contrário fallback localStorage (legado Lovable) onde ainda existir.

4. Backend

Artefato	Caminho
Rotas	backend/internal/interfaces/http/routes/register_chave_digital.go
Handlers	backend/internal/interfaces/http/handlers/
Services	backend/internal/domain/service/

Endpoints principais: GET|POST|DELETE /chave-digital

5. Persistência

Tabelas: chaves_digitais (mig. 000029)

Consultar migrations em backend/migrations/ e ER em [08-banco-dados.md](#).

6. Fluxo operacional

Diagrama (ver Markdown): `flowchart LR...`

7. Sequência — leitura lista

Diagrama (ver Markdown): `sequenceDiagram...`

8. Testes

- Backend: `go test ./internal/domain/service/...`
- Frontend: `*.test.tsx` no domínio, se existir
- E2E: ver `e2e/` para fluxos críticos (ex. agenda em `agenda-sync.spec.ts`)

9. Segurança

- Validar RBAC no guard HTTP antes do handler
- Aplicar data scope por unidade/usuário em services de listagem
- Não expor IDs de outros tenants/unidades na resposta

Apêndice A — Glossário

Termo	Definição
Unidade	Filial clínica (unidades); contexto ativo no header SPA
Consulta	Agendamento clínico (consultas); não confundir com "consulta HTTP"
Terapia	Tipo de tratamento/serviço (ex. fisioterapia); tabela terapias
Profissional	Terapeuta/clinico cadastrado em profissionais
RBAC	Role-Based Access Control — roles + permissões string
Data scope	Filtro automático de linhas por unidade/vínculo do usuário
Soft delete	<code>deleted_at</code> preenchido; registro oculto mas restaurável
Bearer token	JWT enviado em <code>Authorization: Bearer</code>
Feature flag	<code>VITE_API_*</code> controlando integração com backend
Wave 1–3	Ondas de entrega de módulos backend no <code>register.go</code>

Termo	Definição
Prontuário	Registro clínico longitudinal do paciente
Status atendimento	Máquina de estados pós-consulta até aprovação gestor
Chave digital	Par de chaves do usuário para assinatura de documentos
Comodato	Empréstimo de ativos/equipamentos

Apêndice B — Matriz módulo x integração API

Módulo	Flag	Backend	Fallback local
Pacientes	VITE_API_PACIENTES	/pacientes	localStorage (desligado)
Profissionais	VITE_API_PROFISSIONAIS	/profissionais	parcial
Consultas / Agenda	VITE_API_CONSULTAS	/consultas	—
Salas	VITE_API_SALAS	/salas	—
Unidades	VITE_API_UNIDADES	/unidades	seed slugs
Terapias	VITE_API_TERAPIAS	/terapias	—
Anamneses	VITE_API_ANAMNESES	/anamneses	—
Prontuário	VITE_API_PRONTUARIO	/prontuario/*	—
Financeiro	VITE_API_FINANCEIRO	/financeiro/*	—
Contabilidade	VITE_API_CONTABILIDADE	/contabilidade/*	—
Relatórios	VITE_API_RELATORIOS	/relatorios-operacionais	—
Estoque	VITE_API_ESTOQUE	/estoque/*	—
Comodato	VITE_API_COMODATO	/comodatos	—
RH / Folha	VITE_API_RH	/rh/*	—
Planos / Ações / NF	VITE_API_PLANOS	/planos-saude, /acoes-judiciais, /notas-fiscais	—
Contratos	VITE_API_CONTRATOS	/contratos	—
Marketing	VITE_API_MARKETING	/marketing/*	—

Módulo	Flag	Backend	Fallback local
Documentos biblioteca	VITE_API_DOCUMENTOS	/documentos/*	—
Chave digital	VITE_API_CHAVE_DIGITAL	/chave-digital	—
Docs assinados	VITE_API_DOCUMENTOS_ASSINADOS	/documentos-assinados	—
Audit	VITE_API_AUDIT	/audit-log	—
Usuários / RBAC	—	/users, /access-control	—
Exceções agenda prof.	—	—	localStorage em /profissionais/:id/agenda

Regra: em produção, manter todas as flags true exceto testes CI.

Apêndice C — Rotas auxiliares (fora do menu principal)

Rota	Página	Auth	Descrição
/login	Login	Público	Autenticação
/esqueci-senha	EsqueciSenha	Público	Solicita reset
/redefinir-senha	RedefinirSenha	Público	Token e-mail
/alterar-senha	AlterarSenhaObrigatoria	Protegido	Senha temporária
/	Dashboard	Protegido	Home KPIs
/unidades	Unidades	Protegido	Gestão filiais (menu oculto)
/dashboard-ocupacao	DashboardOcupacao	Protegido	Ocupação salas
/prontuario/:consultaId	Prontuario	Protegido	Sessão clínica
/salas/:id/agenda	AgendaSala	Protegido	Grade da sala
/profissionais/:id/agenda	AgendaProfissional	Protegido	Exceções (localStorage)
/relatorios/:id	RelatorioDetalhes	Protegido	Detalhe relatório

Rota	Página	Auth	Descrição
/acoes-judiciais/:id	AcaoJudicialDetalhe	Protegido	Processo judicial
/faturas	Faturas	Protegido	Faturamento
/configuracoes	Configuracoes	Admin	Hub config
/configuracoes/usuarios	Configuracoes	Admin	Usuários
/configuracoes/controles-acesso	ControlesAcesso	Admin	RBAC UI
/configuracoes/chave-digital	ChaveDigital	Admin/Gestor	Chaves
/conta/perfil	ContaPerfil	Protegido	Perfil usuário
/conta/senha	ContaSenha	Protegido	Troca senha
/contratos/compartilhado/:token	ContratoCompartilhado	Público	Link compartilhado
/contratos/assinatura/:token	ContratoAssinatura	Público	Fluxo assinatura

Redirect: /tratamentos → /terapias (compatibilidade Lovable).